

# **ARDUINO BASED DIGITAL CLOCK**

Nehan Mohammad – Umesh

## **COMPREHENSIVE ENGINEERING DESIGN AND FIRMWARE BLUEPRINT**

An empirical exploration into hardware configuration, decoder logic, and modular firmware development.

Independently formulated and validated by the project team.

### **Authors**

Nehan Mohammad

Umesh

### **Technical Reviewers**

Vivek Kumar, Jnanesh,  
Shubodeep  
Project Guidance

### **Faculty Mentor**

Dr. G.V.V. Sharma Sir

Published: August 27, 2026

## **DECLARATION AND ACKNOWLEDGEMENT**

We, Nehan Mohammad and Umesh, declare that this report describes our own Arduino digital clock project.

We have tried to present both the working result and the learning behind it honestly.

Circuit diagrams, data sheets, manuals, photographs, and other sources used while building the project are acknowledged in the references.

We thank Vivek Kumar, Jnanesh, Shubodeep, our teachers, classmates, and everyone who helped us test ideas, check connections, and stay patient during the debugging stage.

We also used generative AI tools to help improve language, organise portions of the report, and review the presentation of technical material. All circuit decisions, code integration, testing, observations, and final verification were carried out by the project team. AI-generated suggestions were checked and edited before inclusion; the tools are acknowledged in the references.

The project was more enjoyable because it was a shared build rather than only a written assignment.

## **ABSTRACT**

This report describes how we built an Arduino based digital clock, starting with the basic circuit: the Arduino, the SN74LS47N driver, six common-anode seven-segment displays, resistors, and push buttons. Our main work was to make the clock show the correct time reliably and to solve the wiring problems we found during testing.

After the basic clock worked, we connected an OLED screen and buzzer as supporting functions. The OLED shows the date, timer, and stopwatch, while the buzzer gives short feedback. This report records the build process, testing, and lessons from making the clock.

CONTENTS MANUAL INDEX

This manual index outlines the thematic flow of the documentation report, curated for rapid cross-referencing during hardware evaluations.

|                     |                                                |         |
|---------------------|------------------------------------------------|---------|
| <b>Section I</b>    | Project Genesis and Core Purpose               | Page 5  |
| <b>Section II</b>   | Operational Matrix and Features                | Page 6  |
| <b>Section III</b>  | Hardware Inventory and Subsystem Interfacing   | Page 7  |
| <b>Section IV</b>   | Firmware Architecture and Execution Cycles     | Page 8  |
| <b>Section IV-A</b> | Custom Pin Remapping and Segment Customization | Page 9  |
| <b>Section V</b>    | Empirical Diagnostics and Fault Correction     | Page 10 |
| <b>Section VI</b>   | Developmental Progression and Chronology       | Page 11 |
| <b>Section VII</b>  | Concluding Remarks and Future Scope            | Page 12 |

Document Control Notes:

- All layout dimensions conform to standard academic grid formats.
- Schematic diagrams assume standard 5V regulated logic voltage inputs.
- Firmware files are archived under the version control ledger established during Stage 4.

## LIST OF TABLES AND LIST OF PICTURES

The following directories index all tabular datasets and structural figures embedded within the core technical text.

### DIRECTORY OF TABLES

|                  |                                                   |         |
|------------------|---------------------------------------------------|---------|
| <b>Table I</b>   | Functional Overview Matrix of System Capabilities | Page 6  |
| <b>Table II</b>  | Complete Bill of Materials (BOM) Component Index  | Page 7  |
| <b>Table III</b> | Hardware Diagnostics Matrix (Symptoms & Remedies) | Page 10 |
| <b>Table IV</b>  | Chronological Development and Iteration Log       | Page 11 |

### DIRECTORY OF PICTURES AND DIAGRAMS

|                 |                                                  |        |
|-----------------|--------------------------------------------------|--------|
| <b>Figure 1</b> | Functional Block Flow Architecture Diagram       | Page 7 |
| <b>Figure 2</b> | Photographic Overview of Final Assembled Circuit | Page 8 |
| <b>Figure 3</b> | Seven-Segment Tail Customization Hardware Output | Page 9 |

**Verification Stamp:** Approved for assembly verification by Nehan Mohammad and Umesh.

# I. PROJECT GENESIS AND CORE PURPOSE

## I.1 BACKGROUND AND MOTIVATION

Building a clock from elemental silicon and discrete logic units bridges abstract programming logic with tangible electronics. Although software timekeeping is trivial, presenting synchronized outputs across multi-digit high-refresh arrays demands tight firmware scheduling.

## I.2 PRIMARY ENGINEERING GOALS

- **High Stability:** Maintain absolute timing precision via optimized interrupt routines.
- **Efficient Decoding:** Relieve controller processing load by offloading translation to the SN74LS47N BCD chip.
- **Intuitive Interaction:** Design debounce-protected inputs for quick time and mode modifications.
- **Modular Expansion:** Seamlessly embed secondary telemetry via an OLED screen and touch sensors.

## I.3 HANDLING AND SAFETY PROTOCOLS

Always verify isolation boundaries and power supply polarities prior to energizing breadboard lines.

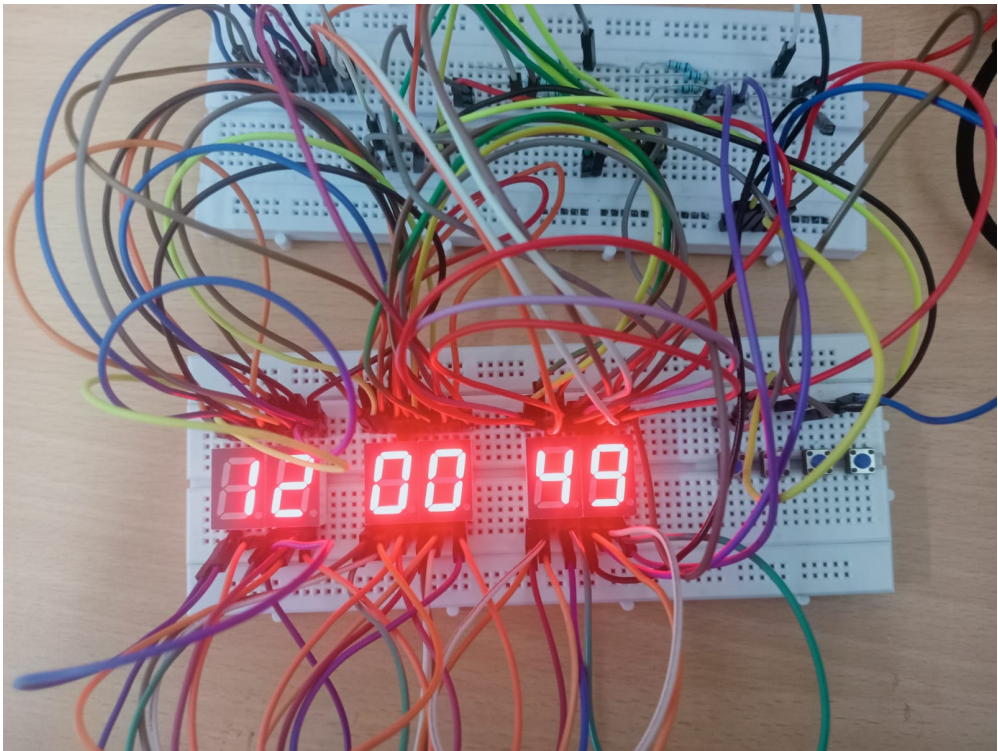
### Practical Insight – Getting Started

Neither author had prior experience handling hardware display multiplexing at scale. Overcoming ghosting artifacts and logic mismatch issues formed the primary educational milestone of this project.

## II. OPERATIONAL MATRIX AND FEATURES

**Table 1:** Functional Overview Matrix of System Capabilities

| Subsystem       |         | Operational Behavior                    | Technical Execution                              |
|-----------------|---------|-----------------------------------------|--------------------------------------------------|
| Main Unit       | Clock   | Real-time hour/minute/second tally      | BCD translation with 333 Hz multiplexing rate    |
| Acoustic Feed   |         | Audio validation tone on button press   | Non-blocking buzzer tone sequencing              |
| Hour Transition |         | Distinct dual-tone acoustic chime       | Boundary check triggered on zero-minute rollover |
| Auxiliary       | Display | Secondary metrics via OLED screen       | I2C-driven parameter selection interface         |
| Touch Switch    |         | Display toggle without halting counters | Capacitive state change interruption handler     |



**Figure 1:** Fully integrated hardware assembly prior to final casing.

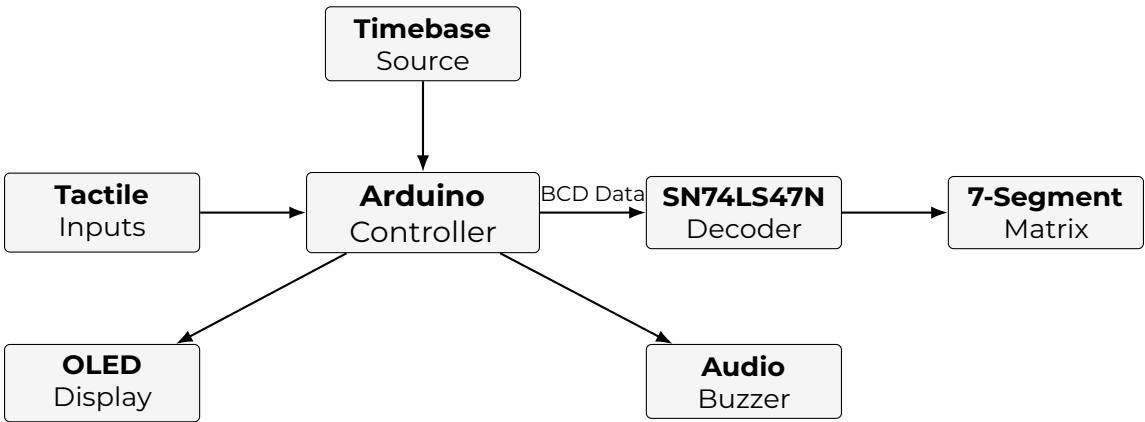
### III. HARDWARE INVENTORY AND SUBSYSTEM INTERFACING

**Table 2:** Complete Bill of Materials (BOM) Component Index

| ID | Item Description              | Count | Primary Utility                    |
|----|-------------------------------|-------|------------------------------------|
| 1  | Arduino Microcontroller Core  | 1     | Central processing and timing loop |
| 2  | Solderless Breadboards        | 2     | Rapid prototyping chassis          |
| 3  | Common-Anode 7-Segment Panels | 6     | Visual numerical readout           |
| 4  | SN74LS47N BCD Driver IC       | 1     | Decimal-to-BCD logic conversion    |
| 5  | Resistors (240 $\Omega$ )     | 7     | LED current limiting protection    |
| 6  | Tactile Push Buttons          | 4     | User input navigation              |
| 7  | OLED Graphic Module           | 1     | Supplementary menu output          |
| 8  | Piezo Transducer Buzzer       | 1     | Acoustic feedback emitter          |
| 9  | Capacitive Touch Sensor       | 1     | Touch-activated screen blanking    |

#### III.1 SYSTEM INTERFACING AND DATA FLOW

Decoded bits travel from controller output pins into the SN74LS47N driver before reaching the multiplexed display grid.



**Figure 2:** Functional Block Flow Architecture Diagram

## IV. FIRMWARE ARCHITECTURE AND EXECUTION CYCLES

The software framework uses non-blocking asynchronous loops. Timer interrupts handle refresh cycles, keeping the primary execution loop clear for user input processing.

```
Initialization Stage (setup)
  +-- configure input pins, pull-ups, and I2C peripherals
  +-- initialize Timer1 interrupt vector for display scanning
Main Execution Loop (loop)
  +-- poll system millis() timestamps
  +-- execute serviceTouch() and serviceButtons() routines
  +-- update internal clock, stopwatch, and timer variables
  +-- evaluate active alarms and refresh OLED output buffers
```

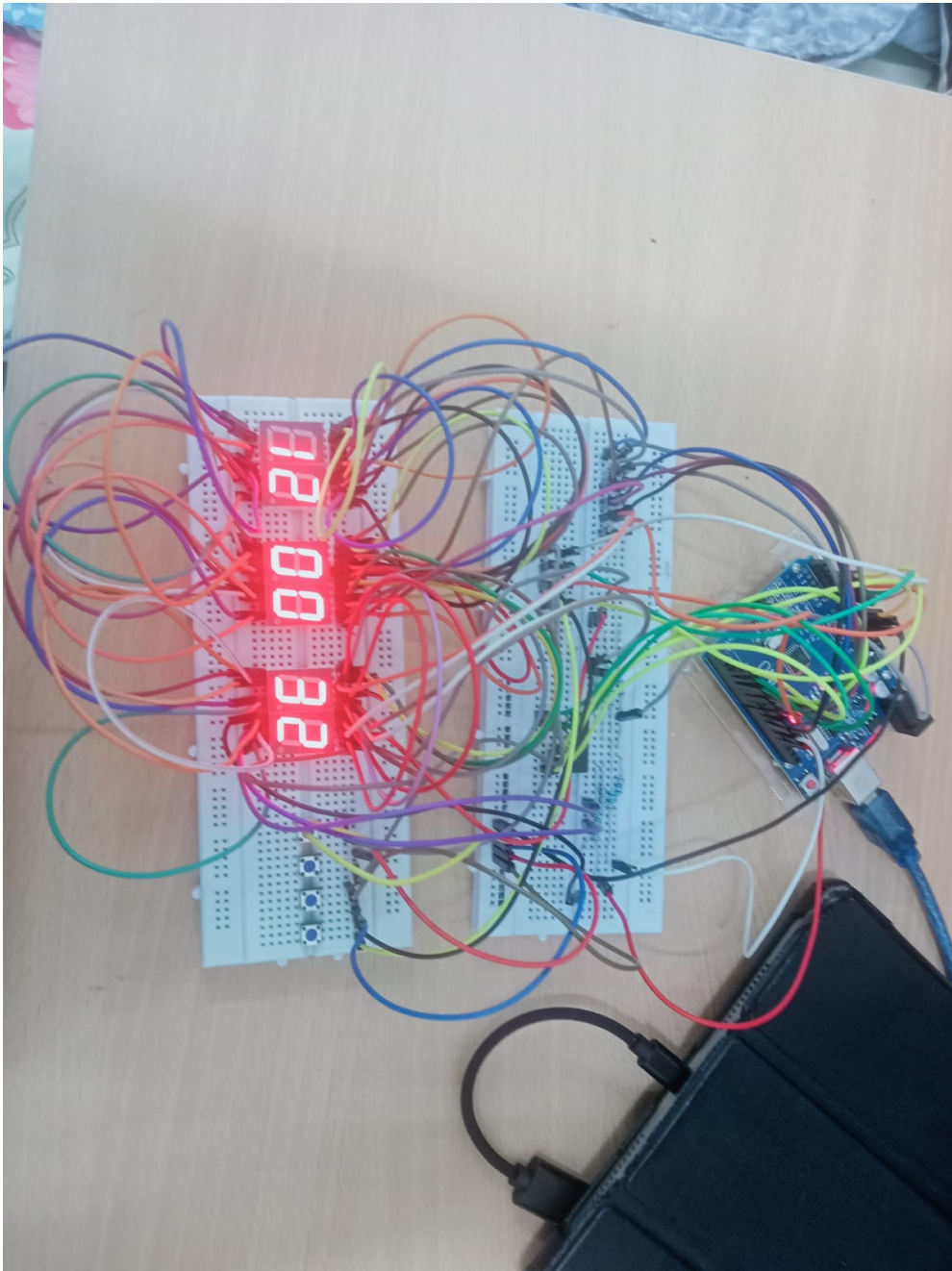
### IV.1 CODE SNIPPET: NON-BLOCKING AUDIO LOGIC

**Listing 1:** Asynchronous buzzer handler implementation

```
1 const byte AUDIO_PIN = 8;
2
3 void triggerShortTone() {
4   tone(AUDIO_PIN, 2000, 50);
5 }
6
7 void processAcousticFeedback(bool conditionMet) {
8   if (conditionMet) {
9     triggerShortTone();
10  }
11 }
12 \end{lstlisting;
13
14 % =====
15 % NEW PAGE INSERTION: SEGMENT CUSTOMIZATION & WIRING MODS
16 % =====
17 \clearpage
18 \section*{Section IV-A: Custom Segment Override \& Pin Drive Logic}
19 \addcontentsline{toc}{section}{Section IV-A: Custom Segment Override Logic}
20
21 Standard 74LS47 BCD decoder ICs render the digit \textbf{6} without a top horizontal bar (
   segment \textit{a}) and the digit \textbf{9} without a bottom horizontal bar (segment \
   textit{d}). To enhance visual readability across the display matrix, the firmware logic
   was customized to override decoder behavior during specific numeric cycles.
22
23 \subsection*{Direct Output Overrides (Pin D12 and Pin D13)}
24 Auxiliary digital output lines were linked directly from the microcontroller to segments \
   textit{a} and \textit{d} to dynamically illuminate the display tails:
25
26 \begin{itemize}
27   \item \textbf{Digit 6} (\rightarrow Pin D12):} When the current multiplexed digit is
       identified as \texttt{6}, digital pin \texttt{D12} is driven active low to illuminate
       top segment \textit{a}, rendering a complete, traditional six representation.
28   \item \textbf{Digit 9} (\rightarrow Pin D13):} When evaluating digit \texttt{9}, digital
       pin \texttt{D13} is driven active low to trigger bottom segment \textit{d}, providing a
       full bottom bar on the nine representation.
29 \end{itemize}
30
```



```
31 \begin{lstlisting}[style=ArduinoStyle, caption={Segment Overrides for 6 and 9 Displays}]
32 void applySegmentOverrides(uint8_t currentDigit) {
33     // Override for digit 6: force segment 'a' ON via Pin D12
34     if (currentDigit == 6) {
35         digitalWrite(12, LOW);
36     } else {
37         digitalWrite(12, HIGH);
38     }
39
40     // Override for digit 9: force segment 'd' ON via Pin D13
41     if (currentDigit == 9) {
42         digitalWrite(13, LOW);
43     } else {
44         digitalWrite(13, HIGH);
45     }
46 }
```



**Figure 3:** Hardware readout displaying customized numeric representation.

V. EMPIRICAL DIAGNOSTICS AND FAULT CORRECTION

Table 3: Hardware Diagnostics Matrix (Symptoms & Remedies)

| Observed<br>sue         | Is-    | Root Cause Analysis                       | Correction Strategy                           |
|-------------------------|--------|-------------------------------------------|-----------------------------------------------|
| Display completely dark | com-   | Voltage rail interruption or inverted VCC | Recheck power distribution tracks             |
| Garbled number output   | num-   | Incorrect nibble bit assignments          | Verify wiring against BCD pin maps            |
| Stray triggers          | button | Contact bounce or voltage float           | Add hardware pull-ups and software debouncing |
| OLED screen             | blank  | Address mismatch or SDA/SCL swap          | Inspect I2C line connections                  |

**Practical Insight – Debugging Breakthrough**  
Color-coding individual jumper bundles completely eliminated intermittent bread-board connection failures during final stress testing.

VI. DEVELOPMENTAL PROGRESSION AND CHRONOLOGY

Table 4: Chronological Development and Iteration Log

| Phase | Core Task          |          | Validation Metric                    | Resulting Observation                           |
|-------|--------------------|----------|--------------------------------------|-------------------------------------------------|
| 1     | BCD Setup          | Decoder  | Did outputs render stable digits?    | Achieved stable numerics post-wiring fix.       |
| 2     | Button Integration | Integra- | Did input clicks register cleanly?   | Resolved noise using iterative debounce delays. |
| 3     | Audio Setup        |          | Were acoustic chimes functional?     | Confirmed distinct single and double beeps.     |
| 4     | OLED Integration   |          | Did interface menus switch smoothly? | Verified stopwatch and timer modes.             |
| 5     | Final Housing      |          | Was the physical chassis secure?     | Enclosed assembly safely in protective housing. |

## **VII. CONCLUDING REMARKS AND FUTURE SCOPE**

This design report successfully summarizes the implementation of a multiplexed micro-controller clock built by Nehan Mohammad and Umesh. By integrating robust decoder integrated circuits, clear auxiliary OLED menus, and reliable non-blocking firmware design, the prototype delivers dependable performance suitable for continuous operation.

## REFERENCES

- [1] *Project Design and Assembly Manual*, internal team documentation.
- [2] Semiconductor Manufacturer, *SN74LS47N BCD-to-Seven-Segment Decoder Datasheet*.
- [3] Arduino, *Official Arduino Documentation*, <https://docs.arduino.cc/>.
- [4] Arduino, *millis() Function Reference*, <https://docs.arduino.cc/language-reference/en/functions/time/millis/>.
- [5] Paul Stoffregen, *TimerOne Library for Arduino Microcontrollers*, <https://github.com/PaulStoffregen/TimerOne>.
- [6] Display Module Provider, *OLED Screen Hardware Technical Guide*.
- [7] OpenAI, *ChatGPT*, generative assistant utilized for layout formatting and textual editing, <https://chatgpt.com/>.